

VELoop Rewards — Full-Stack CAPTCHA Earn Module

Internship Project Assignment

Project: CAPTCHA Earn — Complete MERN Implementation

Task Type: Full-Stack MERN Development

Primary Focus: Backend Architecture + Secure Reward Processing + Frontend Integration + UI/UX

Technology: MERN Stack

Project Assigned On: 22 September 2026

Project Start Date: 22 September 2026

Submission Deadline: 12 October 2026

Submission Time: 5:00 PM IST

1. Project Overview

Build a complete **CAPTCHA Earn Module** for VELoop Rewards.

The feature allows users to solve CAPTCHA-style visual challenges and earn **Gems** based on whether their answer is correct or incorrect.

The original feature flow is:

```
graph TD; A[CAPTCHA] --> B[User views challenge]; B --> C[4 answer options]; C --> D[User selects one]; D --> E[Verification]; E --> F[Checking]; F --> G[Correct / Wrong]; G --> H[Reward]
```

↓
Claim / No Thanks
↓
New CAPTCHA
↓
Continue earning

The original specification requires exactly four answer choices, with one correct option, three reasonably similar-looking options, and one completely different option.

However, **this assignment is NOT frontend-only.**

The intern must convert this concept into a **real MERN application**, where:

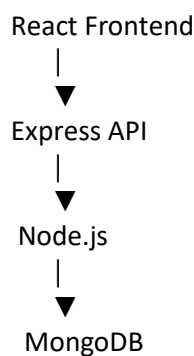
MongoDB + Node.js + Express.js are the source of truth.

The frontend must never be trusted for:

- CAPTCHA correctness
- Reward amount
- User identity
- Gem balance
- Challenge validity
- Challenge completion
- Claim eligibility
- Attempt status
- Reward transaction

2. Main Objective

Build a complete system consisting of:



The user should be able to:

Login
↓
Open CAPTCHA Earn
↓
Receive CAPTCHA from backend
↓
View 4 options
↓
Select answer
↓
Backend validates answer
↓
Checking state
↓
Correct / Wrong result
↓
Backend determines reward
↓
Claim / No Thanks
↓
New CAPTCHA
↓
Continue earning

3. Critical Requirement — No Frontend Cheating

This is one of the **most important requirements of the task**.

The intern must assume that a malicious user can inspect and modify every frontend variable.

Therefore, the frontend must **never contain the actual correct answer or authoritative reward logic**.

❌ **Wrong**

```
const correctAnswer = "A7K2P9";

if (selected === correctAnswer) {
  reward = 1;
}
```

A user can inspect the JavaScript bundle and discover the answer.

❓ **Also wrong**

```
const reward = selected === correct ? 1 : 0.5;
```

❓ **Also wrong**

```
setGems(gems + 1);
```

The frontend cannot decide that the user received a Gem.

4. Correct Architecture

The frontend should receive only the information necessary to display the challenge.

For example:

```
{
  "challengeId": "CAP-92831",
  "question": "A7K2P9",
  "options": [
    "A7K2P9",
    "AJK29P",
    "A7L9P2",
    "X4M8Q1"
  ]
}
```

The **correct answer must NOT be returned to the frontend**.

The backend stores:

```
challengeId
correctOption
reward configuration
status
expiry
userId
```

When the user selects an answer:

```
POST /api/captcha/verify
```

The frontend sends:

```
{  
  "challengeId": "CAP-92831",  
  "selectedOption": "A7K2P9"  
}
```

The backend compares the answer with the database/server-side challenge.

5. CAPTCHA Challenge Generation

The backend must generate/create CAPTCHA challenges.

Each challenge should contain:

```
challengeId  
userId  
captchaText  
options  
correctOption  
status  
expiresAt  
createdAt
```

The actual implementation can use a secure server-side generation strategy.

6. CAPTCHA Format

The CAPTCHA consists of:

- Letters
- Digits

Example:

A7K2P9

The exact generated CAPTCHA should vary.

The original specification intentionally uses character-and-digit combinations such as A7K2P9.

7. Four Answer Options

Every challenge must contain **exactly four options**.

Requirement:

- 1 → Correct
- 2 → Similar incorrect
- 3 → Similar incorrect
- 4 → Completely different

Example:

Correct:
A7K2P9

Options:

A7K2P9	← Correct
AJK29P	← Similar / Wrong
A7L9P2	← Similar / Wrong
X4M8Q1	← Completely Different

The original task explicitly requires this structure.

8. Backend Must Generate the Correct Answer

Do not allow React to generate the authoritative CAPTCHA.

The flow must be:

```
graph TD
    A[React requests challenge] --> B[Backend generates challenge]
    B --> C[Backend stores challenge]
```

↓
Backend sends safe challenge data
↓
React displays it

The backend retains:

correctOption

privately.

9. Challenge Security

Each challenge must be associated with the authenticated user.

Example:

Challenge:
CAP-123

User:
USER-456

User USER-789 must not be able to submit:

CAP-123

as their own challenge.

10. Challenge Expiration

Challenges should have an expiry mechanism.

Example:

Created:
5:00:00 PM

Expires:
5:02:00 PM

The exact duration can be selected by the intern and documented.

Once expired:

verify request
↓
REJECTED
↓
Generate new challenge

This prevents old challenges from being reused indefinitely.

11. Challenge Can Be Used Only Once

Once a challenge has been submitted:

ACTIVE
↓
COMPLETED

It must never be submitted again.

Attempting:

POST /verify

a second time must fail.

Example:

```
{  
  "success": false,  
  "code": "CHALLENGE_ALREADY_COMPLETED",  
  "message": "This CAPTCHA has already been completed."  
}
```

12. Complete User Flow

Step 1 — CAPTCHA Page

User opens:

/captcha-earn

Frontend requests:

GET /api/captcha/current

Backend returns a fresh challenge.

13. CAPTCHA Page UI

The page must display:

- CAPTCHA challenge
- Exactly 4 options
- Gem reward information
- Current relevant earning information
- Supporting UI
- Interactive states
- Premium VELoop design

The original specification says the page should immediately communicate that the activity allows the user to earn Gems.

14. No Generic Radio Buttons

Options must not look like ordinary:

- Option 1
- Option 2
- Option 3
- Option 4

Instead, create premium interactive cards/buttons.

They should respond to:

- Hover
- Touch
- Click

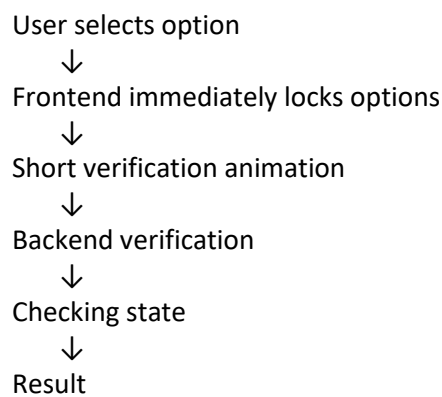
- Selection
- Press
- Focus

The original design specification explicitly calls for interactive options rather than generic radio controls.

15. Selection Flow

There is **NO Submit button**.

The flow must be:



The original requirement specifies automatic processing after selection and no Submit button.

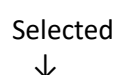
16. Verification Animation

After selection:

Approximately 0.5 seconds

Show a polished verification interaction.

Possible UI:



Scanning



Verifying

Possible effects:

- Selection highlight
- Scanning animation
- Verification icon
- Progress indicator
- Smooth transition

The source specifically calls for an approximately 0.5-second verification animation.

Important

The animation is only a UI experience.

It must **not replace backend verification**.

17. Checking State

After selection, show:

Checking your answer...

with a branded VELoop animation.

The source explicitly requires a dedicated Checking/Verification state rather than a generic browser loading screen.

18. Backend Verification

The backend receives:

```
{  
  "challengeId": "CAP-123",  
  "selectedOption": "A7K2P9"
```

```
}
```

It retrieves:

CAP-123

from MongoDB.

Then:

Is challenge valid?



Does challenge belong to user?



Has challenge expired?



Has challenge already been completed?



Compare selectedOption



Determine result



Determine reward

19. Reward Rules

The existing CAPTCHA specification defines:

Correct Answer

+1 Gem

Wrong Answer

+0.5 Gems

These values are part of the source requirements.

But now, because this is a full-stack system:

These rewards must be controlled by the backend.

Do NOT write:

```
const correctReward = 1;
```

```
const wrongReward = 0.5;
```

as the authoritative reward system in React.

20. Reward Configuration

Create a backend configuration or database structure such as:

CaptchaRewardConfig

Example:

```
correctReward: 1  
wrongReward: 0.5  
currency: GEM  
active: true
```

This makes the reward system configurable.

The frontend receives the result/reward from the API.

21. Correct Result

If the answer is correct:

CORRECT

CAPTCHA verified successfully

+1 Gem

The original specification requires this dedicated rewarding result state.

The reward must come from the backend response.

22. Wrong Result

If the answer is incorrect:

INCORRECT

That answer wasn't correct

+0.5 Gems

The source explicitly defines the wrong-answer reward as +0.5 Gems.

The backend decides this.

23. Gem Wallet Integration

The intern must create a proper backend Gem balance.

Example:

User
↓
Wallet
↓
Gems

Example:

Before:
100 Gems

CAPTCHA:
Correct

Reward:
+1 Gem

After:
101 Gems

The frontend must not perform:

```
gems += 1;
```

as the authoritative update.

24. Gem Ledger

Every CAPTCHA reward must create a transaction/ledger record.

Example:

GemTransaction

```
transactionId
userId
currency = GEM
amount = 1
type = CAPTCHA_REWARD
referenceId = challengeId
balanceBefore = 100
balanceAfter = 101
status = COMPLETED
createdAt
```

For wrong answers:

```
amount = 0.5
```

25. Important Anti-Cheat Rule

A malicious user must not be able to send:

```
{
  "reward": 100000
}
```

and receive 100,000 Gems.

The backend must completely ignore client-supplied reward amounts.

The request should contain only something like:

```
{
  "challengeId": "CAP-123",
  "selectedOption": "A7K2P9"
```

```
}
```

The backend determines everything else.

26. Another Cheat Attempt

User modifies:

```
selectedOption = correctOption;
```

Even if they manipulate the UI, the challenge's actual correct option remains server-controlled.

The backend decides whether the submitted answer is correct.

27. Another Cheat Attempt

User sends:

```
{  
  "challengeId": "CAP-123",  
  "selectedOption": "A7K2P9",  
  "isCorrect": true,  
  "reward": 100  
}
```

Backend must ignore:

```
isCorrect  
reward
```

and independently determine:

```
correct / wrong  
+  
actual reward
```

28. Result API

Example:

POST /api/captcha/verify

Request:

```
{
  "challengeId": "CAP-123",
  "selectedOption": "A7K2P9"
}
```

Response could be:

```
{
  "success": true,
  "result": "CORRECT",
  "reward": {
    "currency": "GEMS",
    "amount": 1
  }
}
```

For wrong:

```
{
  "success": true,
  "result": "WRONG",
  "reward": {
    "currency": "GEMS",
    "amount": 0.5
  }
}
```

29. Claim Button

Both result states must contain:

CLAIM

and:

NO THANKS

as specified in the original flow.

However, because this is now a full-stack task, the intern must implement a **real backend-aware claim flow**, rather than merely creating a visual placeholder.

30. Claim Architecture

The intern should design Claim so that a reward cannot be claimed repeatedly.

Example:

CAPTCHA
↓
Verified
↓
Reward created
↓
Claim available
↓
Claim
↓
Claimed

Once claimed:

Claim again

must be rejected.

31. Claim Idempotency

If the user clicks:

Claim
Claim
Claim

very quickly:

Only one claim/reward transaction may be created.

This must be enforced by the backend.

Frontend button disabling alone is **not sufficient**.

32. Claim Endpoint

Example:

POST /api/captcha/claim

Request:

```
{  
  "challengeId": "CAP-123"  
}
```

Backend verifies:

Challenge belongs to user

Challenge completed

Reward exists

Reward has not already been claimed

Then completes the claim.

33. No Thanks Flow

When the user selects:

NO THANKS

the frontend should request a completely new challenge.

Example:

CAPTCHA #1

↓

Answer

↓

Result

↓

No Thanks

↓

CAPTCHA #2

The original specification explicitly requires the previous CAPTCHA to never be reused.

34. New CAPTCHA API

Example:

GET /api/captcha/new

or:

POST /api/captcha/new

The backend creates a new challenge.

The previous challenge becomes:

COMPLETED

or:

DISCARDED

depending on the state.

35. Previous CAPTCHA Cannot Be Reused

The backend must prevent:

CAPTCHA #1

↓

Completed

↓

Submit CAPTCHA #1 again

This request must fail.

The original specification explicitly requires old CAPTCHA data to be discarded and a new challenge generated after completion.

36. Mock Ad / Claim Experience

The original UI concept includes a future rewarded-ad flow:

Claim
↓
Rewarded Ad
↓
New CAPTCHA

The source specifically says the original frontend task should not integrate a real ad network.

For this **full-stack version**, do not integrate a real third-party ad network either.

Instead create:

Claim
↓
Preparing reward...
↓
Mock Rewarded Ad
↓
Reward completed
↓
New CAPTCHA

This should be clearly a **development/demo state**, not a fake real advertisement.

37. Important Reward Architecture

Do not make the mock ad automatically grant unlimited rewards.

The backend should have a clear state:

rewardStatus:
PENDING
CLAIMED

This prevents:

Claim
→ refresh
→ Claim again
→ another reward

38. Database Models

At minimum, implement:

User
Wallet
CaptchaChallenge
CaptchaAttempt
GemTransaction
CaptchaRewardConfig
AuditLog

You may combine `CaptchaAttempt` and `CaptchaChallenge` if your architecture makes that more appropriate.

39. CaptchaChallenge Model

Recommended:

CaptchaChallenge

- |
- challengeId
- userId
- captchaText
- options
- correctOption
- status
- expiresAt
- selectedOption
- result
- rewardAmount
- rewardStatus

- └─ createdAt
- └─ completedAt

Security note

The `correctOption` must never be exposed through the public API response.

40. CaptchaAttempt Model

If using a separate attempt collection:

CaptchaAttempt

- |
- └─ attemptId
- └─ challengeId
- └─ userId
- └─ selectedOption
- └─ result
- └─ reward
- └─ ip/session metadata where appropriate
- └─ createdAt
- └─ status

This provides better auditability.

41. GemTransaction Model

GemTransaction

- |
- └─ transactionId
- └─ userId
- └─ amount
- └─ type
- └─ source
- └─ referenceId
- └─ balanceBefore
- └─ balanceAfter
- └─ status
- └─ createdAt

Example:

+1 Gem
Source:
CAPTCHA_EARN

Reference:
CAP-123

42. Wallet Balance Must Be Server Authoritative

If the user has:

100 Gems

and earns:

+1

the backend changes:

100 → 101

The frontend then requests the updated wallet.

POST verify/claim

↓

Backend

↓

101 Gems

↓

GET wallet

↓

React displays 101

43. No LocalStorage Wallet

Do not use:

```
localStorage.setItem("gems", "101");
```

as the actual wallet.

A user must not be able to change their balance through browser storage.

44. CAPTCHA APIs

A recommended API structure:

```
GET /api/captcha/current
POST /api/captcha/verify
POST /api/captcha/claim
POST /api/captcha/new
GET /api/captcha/history
GET /api/wallet/gems
GET /api/captcha/config
```

The intern may improve the API architecture.

45. CAPTCHA History

The user should be able to retrieve their own CAPTCHA activity from the backend.

Example:

```
GET /api/captcha/history
```

Data could include:

```
Date
Result
Reward
Status
```

Do not expose:

```
correctOption
```

for completed challenges.

46. Authentication

Use JWT authentication.

Flow:

Login

↓

JWT

↓

Protected CAPTCHA API

↓

Identify User

↓

Create/retrieve challenge

Every challenge must be associated with the authenticated user.

47. User Isolation

User A must not be able to request:

User B's challenge

or:

User B's CAPTCHA history

by modifying a `userId` parameter.

The server should derive identity from authentication.

48. Rate Limiting

Protect:

`/captcha/current`

`/captcha/new`

`/captcha/verify`

`/captcha/claim`

from excessive requests.

For example, a malicious script should not be able to generate thousands of CAPTCHA challenges per second.

The exact limits are up to the intern and must be documented.

49. Abuse Protection

Implement reasonable controls against:

- Challenge replay
- Duplicate claim
- Rapid repeated submissions
- Invalid challenge IDs
- Cross-user challenge access
- Expired challenges
- Automated API spam
- Reward duplication

Do not claim the system is "100% fraud-proof."

50. Transaction Safety

The reward operation should be atomic.

For example:

```
Verify correct
↓
Determine +1 Gem
↓
Create reward transaction
↓
Update Gem balance
↓
Mark challenge completed
```

These operations should not leave the database in an inconsistent state.

Where supported by the MongoDB setup, use MongoDB transactions/atomic operations appropriately.

51. Concurrent Request Test

Suppose a challenge is:

CAP-001

and the user sends:

POST /verify
POST /verify

simultaneously.

The backend must ensure:

Reward:
+1 Gem

NOT:

+2 Gems

This is a mandatory test.

52. Reward Manipulation Test

Send:

```
{  
  "challengeId": "CAP-001",  
  "selectedOption": "correct",  
  "reward": 100000  
}
```

Expected:

Backend ignores reward field.

The user receives only the configured reward.

53. Correct Answer Manipulation Test

Send:

```
{  
  "challengeId": "CAP-001",  
  "selectedOption": "correct",  
  "isCorrect": true  
}
```

The backend must ignore `isCorrect`.

It independently determines whether the answer is correct.

54. Challenge ID Manipulation Test

Try:

CAP-USER-B

while authenticated as User A.

Expected:

403 / 404

and no reward.

55. Expired Challenge Test

Submit an expired challenge.

Expected:

Challenge expired.

Please continue with a new CAPTCHA.

No reward should be created.

56. Completed Challenge Test

Submit an already completed challenge.

Expected:

Challenge already completed.

No second reward.

57. Frontend Design Requirement

Now the frontend should follow the **attached CAPTCHA design/reference**.

The uploaded task specifically establishes the visual direction as:

- Premium
- Modern
- Fintech
- Trustworthy
- Interactive
- Rewarding
- Clean
- Responsive
- Polished.

The intern must recreate the attached design as closely as reasonably possible.

58. Design Must Not Be Generic

Do not build:

- Generic CAPTCHA form
- Plain Bootstrap card
- Generic dashboard
- Basic form
- Random glassmorphism
- Casino UI
- Children's game UI
- Excessively neon UI
- Crypto-style interface

These are specifically contrary to the visual direction in the original task.

59. Design Matching Requirement

The attached design will be placed at the **end of this assignment**.

The intern must reproduce the design's:

- Layout
- Color theme
- Background
- Typography
- Cards
- Borders
- Shadows
- Buttons
- Icons
- Illustrations
- Spacing
- Radius
- Visual hierarchy
- Responsive behavior

Important:

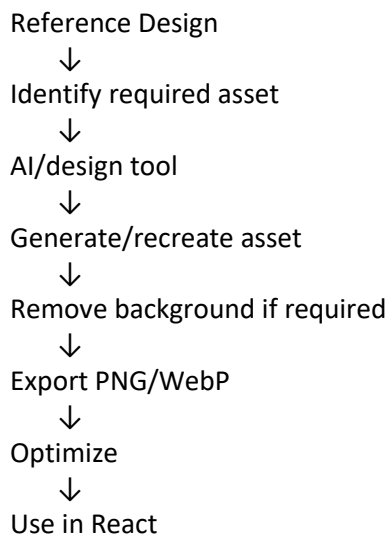
The design is the **visual reference**, while this document defines the **functional/backend requirements**.

60. Assets

If the attached design requires a particular visual asset and that asset is not available separately:

The intern may use an appropriate AI design/image-generation tool to recreate the required visual.

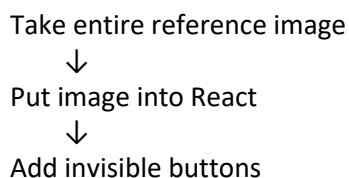
Workflow:



For illustrations that sit directly on the page, transparent PNG/WebP assets are preferred where appropriate.

61. Do Not Use Screenshot-as-UI

This is strictly prohibited:



The UI must be recreated using actual React components.

For example:

CaptchaChallenge.jsx
CaptchaOption.jsx
RewardCard.jsx
CheckingState.jsx
ResultCard.jsx
ClaimButton.jsx

62. Dynamic UI

The page should feel alive.

When CAPTCHA loads:

CAPTCHA reveal

↓

Options enter

↓

Reward information appears

On hover:

Subtle elevation

Border response

Micro animation

On selection:

Immediate selection

↓

Lock options

↓

Verification animation

On result:

Result reveal

↓

Reward animation

The original task explicitly requires dynamic interaction rather than a static interface.

63. Reward Animation

Correct:

+1 Gem

Wrong:

+0.5 Gems

should have a satisfying reveal.

Possible:

- Gem animation
- Counter animation
- Subtle particles
- Success icon
- Smooth scale animation

But it must remain:

Premium fintech/rewards

rather than:

Casino/arcade.

The source makes this distinction explicitly.

64. Responsive Requirements

Support:

320px+

including:

- Mobile
- Tablet
- Laptop

- Desktop
- Large desktop

The original requirements specifically call for mobile-friendly touch interactions, proper spacing, and no horizontal overflow.

65. MERN Technology Stack

Frontend

- React.js
- Vite
- React Router
- CSS Modules
- Bootstrap where appropriate
- React Hooks
- React Icons / Lucide React
- Framer Motion optional

Backend

- Node.js
- Express.js
- MongoDB
- Mongoose
- JWT
- Appropriate validation library
- Appropriate rate-limiting middleware

Deployment

Frontend:

Vercel / Netlify

Backend:

Render / Railway / similar

Database:

MongoDB Atlas

66. Recommended Project Structure

```
veloop-captcha-earn/  
├── frontend/  
│   ├── src/  
│   │   ├── components/  
│   │   ├── pages/  
│   │   │   └── CaptchaEarn/  
│   │   ├── services/  
│   │   ├── hooks/  
│   │   ├── context/  
│   │   ├── assets/  
│   │   └── utils/  
│   └── package.json  
├── backend/  
│   ├── src/  
│   │   ├── config/  
│   │   ├── controllers/  
│   │   ├── middleware/  
│   │   ├── models/  
│   │   ├── routes/  
│   │   ├── services/  
│   │   ├── validators/  
│   │   └── utils/  
│   ├── seed/  
│   └── package.json  
├── docs/  
│   ├── API.md  
│   ├── DATABASE.md  
│   ├── SECURITY.md  
│   └── TESTING.md  
├── postman/  
│   └── VELoop-Captcha.postman_collection.json  
├── README.md  
└── .gitignore
```

67. Suggested Backend Services

```
captcha.service.js  
reward.service.js
```

wallet.service.js
transaction.service.js

Example:

CaptchaService
├── createChallenge()
├── getCurrentChallenge()
├── verifyChallenge()
└── invalidateChallenge()

RewardService
├── calculateReward()
└── grantReward()

WalletService
├── getGemBalance()
└── creditGems()

TransactionService
└── createGemTransaction()

The intern may design a better architecture.

68. Backend Configuration

Do not hardcode everything inside controllers.

Create configurable values for:

Correct reward
Wrong reward
Challenge expiry
Rate limits
Maximum attempts
Challenge configuration

For example:

CaptchaRewardConfig

correctReward = 1
wrongReward = 0.5
currency = GEMS

69. API Documentation

Create:

API_DOCUMENTATION.md

Document at least:

GET /api/captcha/current
POST /api/captcha/verify
POST /api/captcha/claim
POST /api/captcha/new
GET /api/captcha/history
GET /api/wallet/gems

For each:

- Method
 - Authentication
 - Request
 - Response
 - Validation
 - Errors
 - Example
-

70. Postman Collection

Provide a working Postman collection demonstrating:

Login
↓
Get CAPTCHA
↓
Verify Correct
↓
Claim
↓
Get New CAPTCHA
↓
Verify Wrong
↓
Get Gem Balance
↓

Get CAPTCHA History

Also include negative tests.

71. Negative Tests

Include:

Invalid challenge
Expired challenge
Completed challenge
Wrong user
Duplicate claim
Duplicate verification
Invalid option
Missing option
Fake reward
Fake isCorrect
Fake userId
Unauthorized API
Rate limit

72. Production Access

The intern will **NOT** receive VELoop production credentials.

Do not provide:

- Production MongoDB
- Production server
- Production JWT secret
- Production API keys
- Production admin credentials

The intern must build the project independently using:

Their own MongoDB database
Their own deployment
Their own environment variables

73. Demo User

Create a development/demo account.

Example:

Email:
demo@veloop.test

with:

Gem Balance:
100

The actual credentials should be documented safely in the README if required for evaluation.

74. Backend Data Must Be Real

This is **not acceptable**:

```
const captcha = dummyCaptcha;  
const gems = 100;  
const result = true;
```

The evaluator must be able to:

Open MongoDB
↓
See challenge
↓
See attempt
↓
See reward transaction
↓
See updated wallet

75. Frontend Data Must Come From API

The React frontend must obtain:

CAPTCHA
Options
Result
Reward
Gem Balance
History
Challenge Status

from the backend.

No important business value should be permanently embedded in the frontend.

76. Example Complete Flow

Initial balance

Gems:
100

Backend creates:

Challenge:
CAP-001

CAPTCHA:
A7K2P9

Correct option:
Option 2

Frontend receives:

A7K2P9

Option 1
Option 2
Option 3
Option 4

but **not**:

correctOption = Option 2

User selects Option 2

Frontend
↓
POST /verify
↓
Backend
↓
Correct
↓
Reward = 1 Gem
↓
Transaction
↓
Balance 101

77. Wrong Answer Flow

Balance:

101 Gems

User selects incorrect option.

Backend:

Wrong
+
0.5 Gems

Wallet:

101 → 101.5

Ledger:

+0.5
CAPTCHA_REWARD

78. Important Floating-Point Consideration

Because the wrong reward is:

0.5 Gem

the intern should ensure the database/application handles fractional Gems correctly.

Do not rely on unsafe floating-point arithmetic without considering precision.

The intern should document their chosen approach.

79. Claim Flow

Example:

Correct

↓

+1 Gem

↓

Claim

↓

Backend verifies claim state

↓

Reward marked claimed

↓

Mock rewarded-ad state

↓

New CAPTCHA

If the user refreshes after claiming:

Claim again

must not produce another Gem.

80. No Thanks Flow

Result

↓

No Thanks

↓

Old CAPTCHA invalidated



New API request



New CAPTCHA

The old challenge must not be reusable.

81. CAPTCHA History

The backend should preserve completed activity.

Example:

Date	Result	Reward
------	--------	--------

10 Sep	Correct	+1
--------	---------	----

10 Sep	Wrong	+0.5
--------	-------	------

10 Sep	Correct	+1
--------	---------	----

This provides an auditable reward trail.

82. Audit Log

Important actions should be logged:

Challenge Created

Challenge Verified

Reward Created

Reward Claimed

Duplicate Attempt

Invalid Attempt

Expired Challenge

Suspicious Request

This is especially important because the module handles reward credits.

83. README Requirements

README must explain:

Project

VELoop Rewards CAPTCHA Earn

Architecture

React

↓

Express

↓

MongoDB

CAPTCHA generation

Verification

Reward system

Gem wallet

Transaction ledger

Claim system

Security

Anti-cheat architecture

API

Database

Setup

Environment variables

Deployment

Testing

84. Security Documentation

Create:

SECURITY.md

Explain how you prevent:

- Frontend reward manipulation
 - Correct-answer exposure
 - Replay attacks
 - Duplicate rewards
 - Challenge theft
 - User impersonation
 - API abuse
 - Race conditions
-

85. Testing Documentation

Create:

TESTING.md

Include screenshots/results of:

Correct answer
Wrong answer
Claim
No Thanks
New CAPTCHA
Duplicate submission
Expired CAPTCHA
Fake reward
Concurrent request
Unauthorized request

86. GitHub Requirements

Repository must contain:

frontend
backend
docs
postman
README.md
.env.example

Do not commit:

.env
MongoDB credentials
JWT secrets
API keys
private credentials

87. Deployment

A live demonstration is required.

Frontend

Vercel / Netlify

Backend

Render / Railway / equivalent

Database

MongoDB Atlas

The deployed application must actually communicate with the deployed API.

88. Important — No Fake API

This is not accepted:

```
setTimeout(() => {  
  setResult("correct");  
}, 500);
```

with no backend verification.

The UI animation can take approximately 0.5 seconds, but the final result must come from the backend.

89. Important — No Fake Database

This is not accepted:

```
const fakeDatabase = [];
```

or:

```
localStorage
```

as the main persistence mechanism.

MongoDB must actually store the relevant data.

90. Important — Frontend Is Not Trusted

Assume the evaluator will attempt:

- DevTools
- Network tab
- Request modification
- LocalStorage modification
- React state modification
- API replay
- API parameter modification
- Multiple simultaneous requests

The backend must remain authoritative.

91. UI/UX Quality

Although this is a full-stack task, the frontend is still expected to be polished.

The original design standard asks whether the module feels like a legitimate VLoop feature, whether interactions respond naturally, whether rewards feel satisfying, and whether the experience is polished across devices.

Therefore:

Backend quality + frontend quality are BOTH required.

92. What NOT To Build

Do not build:

- Frontend-only CAPTCHA
 - Hardcoded rewards
 - Hardcoded Gem balance
 - Client-side correct answer
 - Fake API
 - Fake database
 - LocalStorage wallet
 - Screenshot-based UI
 - Real ad network
 - Casino-like design
 - Excessive neon
 - Generic Bootstrap form
 - Childish game interface
-

93. Acceptance Criteria

The project will be considered complete only when:

CAPTCHA

- Exactly 4 options
- 1 correct
- 3 reasonably similar incorrect options
- 1 completely different option
- Letters + digits

- Challenge generated by backend
- Correct answer hidden from frontend

Verification

- No Submit button
- Selection automatically triggers verification
- Approximately 0.5-second UI verification animation
- Checking state
- Backend determines result

Rewards

- Correct = +1 Gem
- Wrong = +0.5 Gems
- Backend determines reward
- Gem balance stored in MongoDB
- Ledger transaction created

Claim

- Claim works through backend
- Duplicate claim prevented
- Mock ad state
- New CAPTCHA afterward

No Thanks

- New CAPTCHA
- Old challenge invalidated
- Old challenge cannot be reused

Security

- JWT
- User isolation
- Rate limiting
- Validation
- Duplicate protection
- Replay protection
- Atomic/consistent reward processing
- Server-authoritative rewards

Frontend

- React
- Responsive
- Attached design closely followed
- Premium VELoop style
- Real API integration
- Loading states
- Error states
- Success states

95. Critical Failure Conditions

The project may be rejected if:

- Correct answer is exposed in the frontend.
- Reward is calculated only in React.
- Gem balance is controlled by React.
- LocalStorage is used as the wallet.
- CAPTCHA can be replayed.
- Same CAPTCHA can generate multiple rewards.
- User can submit another user's challenge.
- User can manipulate reward amount.
- User can manipulate `isCorrect`.
- Backend isn't actually connected to MongoDB.
- APIs are fake/mock-only.
- UI is just a screenshot.
- No GitHub repository.
- No working live demonstration.
- Production credentials are committed.

96. Final Challenge

After completing the project, answer:

"If VELoop Rewards receives 100,000 CAPTCHA attempts per day, how would you scale this system while preventing duplicate rewards, replay attacks, automated abuse and inconsistent Gem balances?"

Discuss:

- MongoDB indexes
 - Atomic updates
 - Transactions
 - Idempotency
 - Rate limiting
 - Challenge expiry
 - Redis where appropriate
 - Queue architecture
 - Monitoring
 - Fraud/risk signals
 - Database scaling
 - API scaling
 - Reward reconciliation
-

97. Final Submission Checklist

Before submitting:

Backend

- ☐ Node.js
- ☐ Express.js
- ☐ MongoDB
- ☐ Mongoose
- ☐ JWT authentication
- ☐ CAPTCHA generation
- ☐ Server-side answer verification
- ☐ Challenge expiry
- ☐ Challenge invalidation
- ☐ Replay prevention

- ☐ Reward configuration
- ☐ Gem wallet
- ☐ Gem ledger
- ☐ Claim API
- ☐ Duplicate claim protection
- ☐ Rate limiting
- ☐ Validation
- ☐ Audit logs

Frontend

- ☐ React
- ☐ Vite
- ☐ Real backend API integration
- ☐ CAPTCHA UI
- ☐ Four options
- ☐ Selection animation
- ☐ Checking state
- ☐ Correct state
- ☐ Wrong state
- ☐ +1 Gem display
- ☐ +0.5 Gem display
- ☐ Claim
- ☐ No Thanks
- ☐ New CAPTCHA
- ☐ Loading state
- ☐ Error state
- ☐ Responsive design
- ☐ Attached design closely implemented

Testing

- ☐ Correct answer

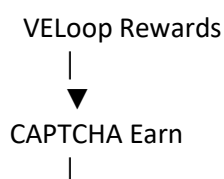
- ☐ Wrong answer
- ☐ Expired challenge
- ☐ Duplicate verification
- ☐ Duplicate claim
- ☐ Fake reward
- ☐ Fake isCorrect
- ☐ User isolation
- ☐ Concurrent requests
- ☐ API abuse/rate limit

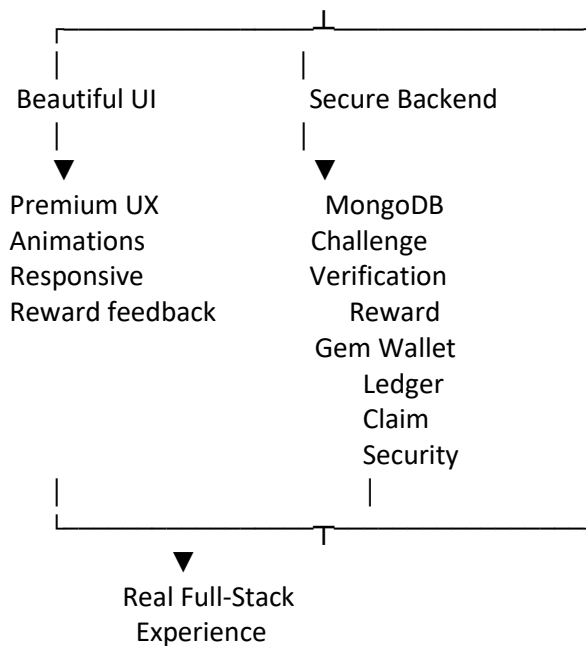
Submission

- ☐ GitHub repository
- ☐ Live frontend
- ☐ Live backend
- ☐ MongoDB development database
- ☐ README
- ☐ API documentation
- ☐ Database documentation
- ☐ Security documentation
- ☐ Testing documentation
- ☐ Postman collection
- ☐ .env.example
- ☐ Demo video

98. Project Quality Bar

The final product should feel like:





The original task's final objective is to transform a normally boring CAPTCHA activity into a smooth, premium VELoop earning experience.

For this revised assignment, the same experience must now be backed by a real MERN architecture where the user cannot cheat the CAPTCHA, manipulate rewards, replay challenges, or directly alter their Gem balance through the frontend.

99. DESIGN REFERENCE

ATTACHED DESIGN — IMPLEMENT THE CAPTCHA EARN PAGE IN THIS SAME DESIGN LANGUAGE

The design attached at the end of this task is the visual reference.

The intern must:

1. Study the complete design before coding.
2. Identify all UI components.
3. Identify the color palette.
4. Identify typography.
5. Identify illustrations/assets.
6. Recreate the UI using React.

7. Maintain the same visual hierarchy.
8. Maintain the same overall color theme.
9. Make every UI element functional.
10. Connect the UI to the MERN backend.
11. Test it on mobile, tablet and desktop.
12. Polish animations and interactions before submission.

If any individual illustration is required and unavailable as a separate asset, use an appropriate AI/design tool to recreate the **individual asset**, export it as PNG/WebP with transparent background where appropriate, and integrate that asset into the React UI.

Do not use the attached design as a single screenshot/background.

FINAL OBJECTIVE

The intern should be able to demonstrate:

User Login
↓
CAPTCHA Earn
↓
Backend generates unique challenge
↓
Frontend displays 4 options
↓
User selects answer
↓
0.5 sec premium verification animation
↓
Backend validates answer
↓
Correct / Wrong
↓
Backend determines reward
↓
Gem transaction created
↓
Gem balance updated
↓
Claim / No Thanks
↓
Challenge permanently closed
↓
Fresh CAPTCHA generated



User continues earning

The key principle is:

The frontend displays the experience. The backend owns the truth.

And the final UI should make the user feel:

“This feels like a real, premium VELoop Rewards earning feature—not a basic CAPTCHA form.”

DESIGN ATTACHED BELOW BUILD SAME AS THAT

CAPTCHA EARN FLOW

Secure Verification • Earn Rewards • Build Trust

1 CHALLENGE

9:41

VELOOP REWARDS 125.50

Earn Gems

Complete a quick security check to earn rewards.

A 7 K 2 P 9 New Code

Select the matching code

A7K2P9

AJK29P

A7L9P2

X4M8Q1

This helps protect your account from automated access.

Complete verification to earn +1 Gem

2 OPTION SELECTED

9:41

VELOOP REWARDS 125.50

Earn Gems

Complete a quick security check to earn rewards.

A 7 K 2 P 9 New Code

Select the matching code

A7K2P9

AJK29P

A7L9P2

X4M8Q1

Submit Answer

This helps protect your account from automated access.

Complete verification to earn +1 Gem

3 VERIFYING

9:41

VELOOP REWARDS 125.50

Verifying...

Please wait while we check your answer.

Do not close this screen while verification is in progress.

4 SUCCESS

9:41

VELOOP REWARDS 126.50

Verification Complete!

You earned +1 Gem

125.50 Previous Balance

→

126.50 New Balance

Add to Balance

Maybe Later

Your reward has been added to your account.

5 INCORRECT

9:41

VELOOP REWARDS 125.50

Verification Unsuccessful

The selected code doesn't match the image shown.

Please try again with a new challenge.

Try Again

Get New Code

Security checks keep your account safe.

SECURE
Advanced protection for your account

REWARDING
Earn gems for completing verification

FAST
Quick verification and rewards

MOBILE FIRST
Optimized experience on every device

TRUSTED
Your security is our priority